



Ethical Aspects of Synthetic Data in Smart Meter Datasets

A feature-based privacy comparison of a real (SmartNIALMeter) and a synthetic (SynD) household energy dataset

Smart Grid Research Project · 700.288 · Summer Semester 2026

Submitted by

Ucho Ikechukwu Igwe

Matriculation No. 12349300

Submitted to

Prof. Wilfried Elmenreich
Fabian Schober (co-supervisor)

University of Klagenfurt, Austria

Table of Contents

Abstract 3

1. Introduction and Scope..... 4

2. Datasets 4

3. Methodology..... 4

 3.1 Data acquisition..... 4

 3.2 Reducing the data to a manageable size 5

 3.3 Feature extraction..... 5

 3.4 Privacy metrics 6

4. Results..... 7

 4.1 What the load profiles look like 7

 4.2 Privacy metrics 8

5. Bias: a brief qualitative review 9

6. Conclusion 9

References 10

Appendix A. Complete analysis pipeline..... 11

Abstract

Smart meters record household electricity usage at fine time resolution. That same resolution makes the data personal: a load curve reveals when people wake, cook, leave, and sleep. Synthetic data is often proposed as a privacy preserving substitute, but the claim that synthetic data is “private” is rarely tested. In this project I evaluate it directly. I take one real dataset SmartNIALMeter, SNM from 20 Swiss homes and one synthetic dataset SynD from one simulated Austrian household, reduce each daily load curve to a vector of eight behavioral features, and compute feature-based privacy metrics on each dataset independently. The central test, suggested by my supervisor, is single day re-identification: can one ordinary day be matched back to the household it came from? Working on four real homes and the synthetic household, I find a sharp difference. A single day of a real home identifies the correct household 77.2% of the time (chance = 25%), and the real homes carry a composite Privacy Risk Score of 0.89. The synthetic days cannot be linked at all (re-identification 0.000, PRS 0.29). I argue that SynD is private not through deliberate anonymization but as a by-product of how it is generated, that is independent days and flat overnight behavior and I summarize the whole evaluation in a single portable privacy score.

1. Introduction and Scope

Non-Intrusive Load Monitoring (NILM), introduced by Hart in 1992 [3], infers which appliances run in a home from the single aggregate signal at the main meter. NILM needs data, and collecting it in real homes is expensive and intrusive, which is the main motivation for synthetic datasets. The ethical question this project addresses is whether the synthetic alternative is genuinely more private than the real data it replaces and whether that can be shown with evidence rather than assumed.

After supervisor feedback I deliberately narrowed an earlier, broader plan into a single self-contained study with one clear question and two datasets. The question: how identifiable is an individual household, and does the synthetic dataset carry less of that risk than the real one?

I do not compare the two datasets record by record because they contain different households, so a record to record distance such as NNDR is meaningless here). I Instead evaluate each dataset independently on the same feature-based metrics and compare the resulting numbers.

2. Datasets

	SmartNIALMeter (SNM) Real [1]	SynD Synthetic [2]
Origin	Measurement in 20 Swiss homes (Zürich, Bern, Lucerne)	Simulation from a campaign in 2 Austrian homes
Mains rate	Site meter (L+G E450) every 5 s (0.2 Hz)	5 Hz, aggregate of appliances
Duration	Up to ~2 years per home	180 days, one virtual household
Stored as	V, I, power factor per phase (HDF5)	Active power in W (CSV / HDF5)
This study uses	4 homes (b10, b11, b15, b17)	the single household

Both dataset papers [1][2] are also the primary literature for the project, as the original exposé suggested.

3. Methodology

This section documents every step I actually took, from raw download to final metric, including the commands and Python code used. The work was done on Windows 10 with Python 3.12 (pandas, numpy, matplotlib, PyTables) and 7-Zip for archive handling.

3.1 Data acquisition

The two datasets are hosted on different repositories and were downloaded manually:

- SynD: the CSV release (SynD_CSV.zip, 5.2 GB) from the figshare collection [2]. Inside, channel 1 (file 1.csv) is the whole home mains signal at 5 Hz, tab-separated, timestamp + power in watts.
- SNM: the raw archive (raw.7z, 10 GB) from Zenodo record 10875988 [1]. Each building is a folder; the mains is the site-meter file cii-adapter.h5.

Because neither archive fits comfortably on the working machine when fully extracted, I never unpacked them whole. I used 7-Zip to list the archive contents and then extract only the required mains files:

```
# list the SNM archive without extracting (no disk cost)
"C:\Program Files\7-Zip\7z.exe" l raw.7z > snm_list.txt

# extract ONLY the mains (site-meter) file of selected buildings
"C:\Program Files\7-Zip\7z.exe" x raw.7z -osnm_mains \
  "raw\building_10\cii-adapter.h5" "raw\building_11\cii-adapter.h5" \
  "raw\building_15\cii-adapter.h5" "raw\building_17\cii-adapter.h5"
```

Listing 1. Selective extraction of mains files with 7-Zip, avoiding a full ~45 GB unpack.

3.2 Reducing the data to a manageable size

The privacy metrics operate on per-day behavioral features, not on raw samples, so the first processing step is to resample each mains signal to one minute averages. For SynD this shrinks a ~78-million-row, 3 GB CSV to 259,200 rows (~10 MB). Because the raw file is too large to hold in memory of my PC, it is read in chunks:

```
import pandas as pd
# SynD mains: tab-separated, no header -> 1-minute averages
parts = []
for chunk in pd.read_csv("1.csv", sep="\t", header=None,
                        names=["time", "power_w"], chunksize=2_000_000):
    chunk["time"] = pd.to_datetime(chunk["time"], errors="coerce")
    chunk = chunk.dropna(subset=["time"]).set_index("time")
    parts.append(chunk["power_w"].resample("1min").mean())
s = pd.concat(parts).groupby(level=0).mean()
s.to_csv("synd_mains_1min.csv", header=["power_w"])
```

Listing 2. Chunked downsampling of the SynD mains signal to one-minute resolution.

The SNM mains needed an extra step. The raw site-meter file does not store power directly; it stores RMS voltage, RMS current and power factor for each of the three phases (L1, L2, L3). I therefore computed real power per the dataset paper's own definition, $P = V \cdot I \cdot \cos\phi$, and summed the three phases to obtain whole home power, then resampled to one minute and clipped obvious sensor spikes:

```
import pandas as pd
df = pd.read_hdf("cii-adapter.h5", "data") # columns: Voltage/Current/Power Factor L1..L3
df.index = pd.to_datetime(df.index)
P = 0.0
for ph in ["L1", "L2", "L3"]:
    P = P + df["Voltage "+ph] * df["Current "+ph] * df["Power Factor "+ph]
P = P.clip(lower=0, upper=P.quantile(0.999)) # drop negatives and rare measurement spikes
out = P.resample("1min").mean().dropna()
out.to_csv("snm_bXX_mains_1min.csv", header=["power_w"])
```

Listing 3. Computing whole-home power for an SNM building from per-phase V/I/PF and resampling.

3.3 Feature extraction

Each day of one minute mains data is reduced to a fixed vector of eight behavioral features. These were chosen because each has a clear human meaning and because the re-identification literature treats such timing and magnitude summaries as identifying [4]:

Feature	Meaning
total_kwh	energy consumed that day

Feature	Meaning
peak_w / mean_w	highest and average power
load_factor	mean / peak (how “peaky” the day is)
morning_rise_h	hour activity first exceeds the daily baseline (wake-up proxy)
evening_peak_h	hour of the evening maximum
night_day_ratio	energy 00:00–06:00 vs the rest of the day (occupancy / away signal)
n_peaks	number of distinct activity bursts

```

FEATURES = ["total_kwh", "peak_w", "mean_w", "load_factor",
            "morning_rise_h", "evening_peak_h", "night_day_ratio", "n_peaks"]

def day_features(d):
    # d = one day of 1-min power (W)
    p = d.values; hrs = d.index.hour + d.index.minute/60
    peak, mean = p.max(), p.mean()
    base = np.median(p); thr = base + 0.5*(peak-base)
    above = (p>thr) & (hrs>=4)
    morning = hrs[np.argmax(above)] if above.any() else np.nan
    ev = hrs>=16; evening = hrs[ev][np.argmax(p[ev])] if ev.any() else np.nan
    nmask = (hrs>=0)&(hrs<6)
    ndr = p[nmask].sum() / p[~nmask].sum()
    sig = (p>thr).astype(int); n_peaks = int(((sig[1:]-sig[:-1])==1).sum())
    return dict(total_kwh=p.sum()/60/1000, peak_w=peak, mean_w=mean,
               load_factor=mean/peak, morning_rise_h=morning,
               evening_peak_h=evening, night_day_ratio=ndr, n_peaks=n_peaks)

```

Listing 4. Behavioral feature extraction from one day of mains power.

3.4 Privacy metrics

On the feature tables I compute five feature-based metrics. For the real data, quasi-identifier uniqueness and k-anonymity are computed across the four households (each home reduced to its mean feature profile). For SynD, which is a single household, the same metrics are computed across its 180 days, treating each day as a record which is the natural analogue of “how distinguishable is one unit from the others.”

```

def qi(frame, cols, bins=4):
    # quasi-identifier = quantile-binned feature tuple
    b = frame[cols].apply(lambda c: pd.qcut(c.rank(method="first"),
        q=min(bins,len(frame)), labels=False, duplicates="drop"))
    return b.astype(str).agg("|".join, axis=1)

def uniqueness_entropy_k(frame):
    q = qi(frame, ["morning_rise_h", "evening_peak_h", "night_day_ratio", "total_kwh"])
    cls = q.value_counts(); n = len(q)
    uniqueness = cls[cls==1].sum()/n # share with a one-of-a-kind signature
    pr = cls.values/n; H = -(pr*np.log2(pr)).sum()
    entropy_norm = H/np.log2(len(cls))
    k = int(cls.min()) # k-anonymity
    return uniqueness, entropy_norm, k

```

Listing 5. Feature uniqueness, normalized entropy and k-anonymity.

The single day re-identification test is the centerpiece. For the real homes: hold out one day, build each household’s profile from its remaining days, and check whether the held out day’s nearest profile is its own household. The score is top -1 accuracy; random chance is 1/(number of homes).

```
def single_day_reid(frame, cols=FEATURES):
    X = frame[cols].values; mu, sd = X.mean(0), X.std(0)+1e-9
    correct = 0
    for i in range(len(frame)):
        # leave-one-day-out
        true = frame.household.iloc[i]; profiles = {}
        for h in frame.household.unique():
            m = (frame.household.values==h); m[i] = False
            profiles[h] = ((X[m]-mu)/sd).mean(0) # profile from that home\2019s OTHER days
        q = (X[i]-mu)/sd
        pred = min(profiles, key=lambda h: np.linalg.norm(q-profiles[h]))
        correct += (pred==true)
    return correct/len(frame), 1/frame.household.nunique()
```

Listing 6. Single day household re-identification (leave-one-day-out nearest profile).

Finally, the five metrics are folded into one Privacy Risk Score (PRS) on a 0–1 scale, weighting the single day re-identification and uniqueness most heavily because they map most directly to real world harm.

```
def privacy_risk_score(uniqueness, entropy_norm, k, reid):
    k_term = 1 - min(k,10)/10 # small k -> higher risk
    return (0.40*reid + 0.30*uniqueness + 0.20*k_term + 0.10*entropy_norm)
```

Listing 7. Composite Privacy Risk Score (the portable “meta-metric”).

4. Results

4.1 What the load profiles look like

The synthetic and real homes already differ at the level of the average day (Figure 1). SynD is dead overnight and only rises after noon. The real Swiss homes draw far more power, show genuine overnight activity, and each has its own shape.

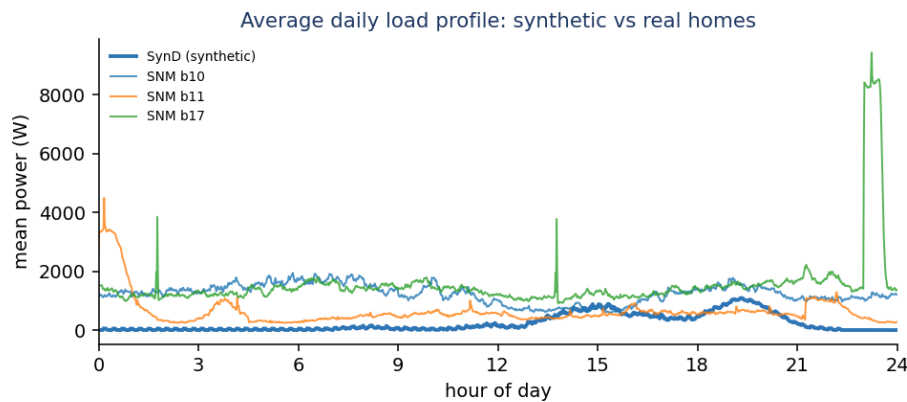


Figure 1. Average daily load profile: the synthetic household (blue) vs three real SNM homes.

The four real homes are also clearly distinct from one another, which is what makes them identifiable:

Home	Days	Daily kWh	Mean peak (W)	Wake (h)	Eve. peak (h)	Night/day
b10 (real)	189	29.8	8514	8.5	18.9	0.41
b11 (real)	210	15.2	6566	10.0	19.0	0.63
b15 (real)	72	60.6	38977	8.8	19.2	1.31

Home	Days	Daily kWh	Mean peak (W)	Wake (h)	Eve. peak (h)	Night/day
b17 (real)	236	36.7	10982	10.8	23.1	0.26
SynD (synthetic)	180	5.8	3813	12.6	17.4	0.03

Two things stand out. The real homes consume far more and have real overnight activity (night/day ratios 0.26–1.31), while SynD is almost dormant at night (0.03). And SynD has no morning: its first activity averages 12.6 h, against 8.5–10.8 h for the real homes.

4.2 Privacy metrics

The metrics make the contrast quantitative.

Metric	SynD (synthetic)	SNM (real)
Records evaluated	180 days	4 homes / 4 days
Feature uniqueness	0.044	1.000
Normalized entropy	0.972	1.000
k-anonymity	1	1
Single-day re-identification	0.000	0.772
random baseline	0.0056	0.25
Privacy Risk Score (0–1)	0.291	0.889

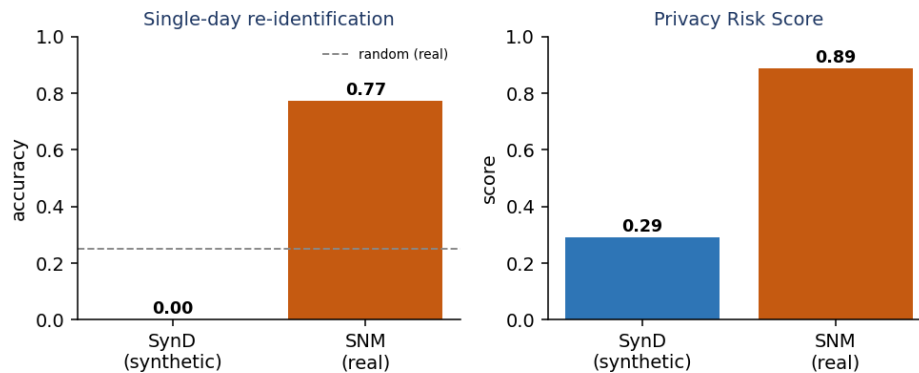


Figure 2. Single day re-identification and composite Privacy Risk Score: real vs synthetic.

The decisive result is single day re-identification. A single day of a real household is matched back to the correct home 77.2% of the time, far above the 25% you would get by guessing. The same test on SynD returns 0.000 which is no better than chance. The composite Privacy Risk Score summarizes the gap: 0.89 for the real home vs 0.29 for SynD.

The behavioral reason is visible in the overnight signal (Figure 3): real homes spread across a wide range of night/day ratios, each home's nightly habits are part of its fingerprint whereas SynD's nights are uniformly flat.

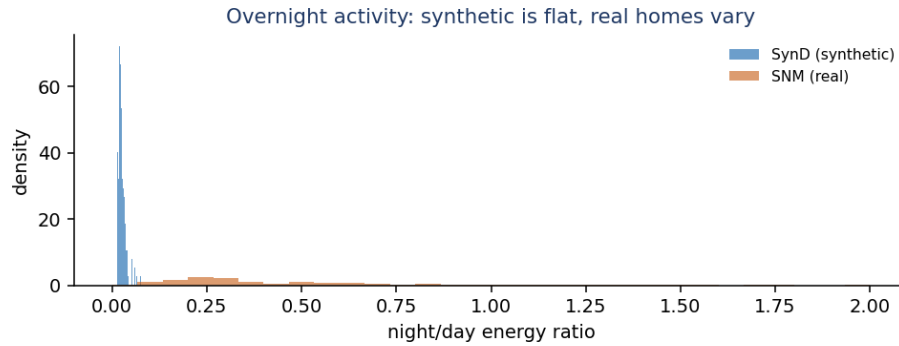


Figure 3. Night/day energy ratio. Real homes vary widely; the synthetic household is uniformly low.

Caveat on k-anonymity. Both datasets show $k = 1$, but at these sample sizes (4 homes; 180 days) and with continuous features, $k = 1$ is expected rather than alarming, almost any continuous profile is unique once. I therefore treat uniqueness and single day re-identification as the trustworthy readings and flag k-anonymity as informative only at larger population sizes.

5. Bias: a brief qualitative review

Following the feedback I got, I keep the bias analysis short and qualitative, based on dataset metadata rather than a population survey. Both datasets describe relatively affluent, electrified households. Across the 20 SNM homes the appliance metadata shows washing machines in 16, fridges and freezers in 15, but heat pumps in only 4 and EV charging in 2 shows a comfortable, technically engaged profile typical of volunteer studies in wealthy Swiss cities. SynD inherits a similar skew: its appliance list like a branded coffee machine, a gaming PC, a laptop describes a particular lifestyle, and because it was simulated from two Austrian homes and tuned with distributions from another dataset, it copies whatever bias those sources held rather than correcting it. Synthetic generation does not fix representation bias; it propagates it.

6. Conclusion

The evidence is consistent and points one way: on feature-based privacy metrics the real homes are highly identifiable and the synthetic household is not. But the interesting part is the reason. The SynD paper states that the generator builds each day independently and draws appliance start times from fixed probability windows [2]. My measurements are the empirical signature of exactly that design that is no day-to-day link, near constant overnight behavior, and timing that barely varies. So SynD's low privacy risk is a by-product of behavioral flatness and day independence, not of any deliberate anonymization.

This matters for how synthetic data should be treated. A synthetic dataset can score as “private” simply because it under models real human routine that is the same property that limits its realism for some NILM tasks. Privacy and fidelity are in tension, and a privacy claim should be demonstrated with metrics like these rather than assumed from the word “synthetic.”

The single Privacy Risk Score gotten 0.29 synthetic vs 0.89 real is the portable contribution: because it is defined only over extracted features, the same number can be computed on any smart-meter dataset to give a comparable, at-a-glance risk figure before publication.

My Limitations: I used four SNM homes, not all twenty, because of my pc disk-space constraints during processing; the re-identification baseline is correspondingly 25%, and a larger pool would strengthen the estimate. SynD is a single household, so its metrics are day level and not strictly identical in construction to the household level metrics for SNM. And SynD's mains is the noiseless sum of its appliances [2], which may make it look cleaner than a real meter. None of these change the direction of the result, but they bound how far it should be pushed.

References

- [1] M. Vogel, M. Friedli, M. Camenzind, et al., "The 'SmartNIALMeter' electrical appliance disaggregation dataset," *Data in Brief*, vol. 56, p. 110854, 2024. doi:10.1016/j.dib.2024.110854.
- [2] C. Klemenjak, C. Kovatsch, M. Herold, and W. Elmenreich, "A synthetic energy dataset for non-intrusive load monitoring in households," *Scientific Data*, vol. 7, no. 1, p. 108, 2020. doi:10.1038/s41597-020-0434-6.
- [3] G. W. Hart, "Nonintrusive appliance load monitoring," *Proc. IEEE*, vol. 80, no. 12, pp. 1870–1891, 1992.
- [4] E. Buchmann, K. Böhm, T. Burghardt, and S. Kessler, "Re-identification of smart meter data," *Personal and Ubiquitous Computing*, vol. 17, no. 4, pp. 653–662, 2013.
- [5] L. Sweeney, "k-anonymity: a model for protecting privacy," *Int. J. of Uncertainty, Fuzziness and Knowledge-Based Systems*, vol. 10, no. 5, pp. 557–570, 2002.
- [6] European Parliament and Council, "Regulation (EU) 2016/679 (General Data Protection Regulation)," *Official Journal of the EU*, 2016.

Appendix A. Complete analysis pipeline

The full Python program below reproduces every result in this report from the prepared one minute mains CSVs (one per dataset). It is the consolidated form of Listings 4–7.

```
import pandas as pd, numpy as np
np.random.seed(0)
FEATURES = ["total_kwh", "peak_w", "mean_w", "load_factor",
            "morning_rise_h", "evening_peak_h", "night_day_ratio", "n_peaks"]

def load(path):
    df = pd.read_csv(path); df.columns = ["time", "power_w"][:len(df.columns)]
    df["time"] = pd.to_datetime(df["time"])
    return df.set_index("time")["power_w"].clip(lower=0)

def day_features(d):
    if len(d) < 600: return None
    p = d.values; hrs = d.index.hour + d.index.minute/60
    peak, mean = p.max(), p.mean()
    if peak <= 0: return None
    base = np.median(p); thr = base + 0.5*(peak-base)
    above = (p>thr) & (hrs>=4)
    morning = float(hrs[np.argmax(above)]) if above.any() else np.nan
    ev = hrs>=16; evening = float(hrs[ev][np.argmax(p[ev])]) if ev.any() else np.nan
    nmask = (hrs>=0)&(hrs<6); en, ed = p[nmask].sum(), p[~nmask].sum()
    sig = (p>thr).astype(int); npk = int(((sig[1:]-sig[:-1])==1).sum())
    return dict(total_kwh=p.sum()/60/1000, peak_w=peak, mean_w=mean,
               load_factor=mean/peak, morning_rise_h=morning,
               evening_peak_h=evening if ev.any() else np.nan,
               night_day_ratio=en/ed if ed>0 else 0, n_peaks=npk)

def extract(path, hid):
    s = load(path); rows = []
    for day, d in s.groupby(s.index.normalize()):
        f = day_features(d.dropna())
        if f: f["household"]=hid; rows.append(f)
    return pd.DataFrame(rows).dropna(subset=FEATURES)

def qi(frame, cols, bins=4):
    b = frame[cols].apply(lambda c: pd.qcut(c.rank(method="first"),
        q=min(bins,len(frame)), labels=False, duplicates="drop"))
    return b.astype(str).agg("|".join, axis=1)

def uniqueness_entropy_k(frame, household_level):
    qc = ["morning_rise_h", "evening_peak_h", "night_day_ratio", "total_kwh"]
    base = frame.groupby("household")[qc].mean().reset_index() if household_level else frame
    q = qi(base, qc); cls = q.value_counts(); n = len(q)
    u = cls[cls==1].sum()/n; pr = cls.values/n; H = -(pr*np.log2(pr)).sum()
    return u, (H/np.log2(len(cls)) if len(cls)>1 else 0.0), int(cls.min())

def single_day_reid(frame, cols=FEATURES):
    arr = frame.reset_index(drop=True); hh = arr.household.unique()
    X = arr[cols].values; mu, sd = X.mean(0), X.std(0)+1e-9; c = 0
    for i in range(len(arr)):
        true = arr.household[i]; prof = {}
        for h in hh:
            m = (arr.household.values==h).copy(); m[i] = False
            if m.any(): prof[h] = ((X[m]-mu)/sd).mean(0)
        q = (X[i]-mu)/sd
        c += (min(prof, key=lambda h: np.linalg.norm(q-prof[h]))==true)
    return c/len(arr), 1/len(hh)

def prs(u, Hn, k, reid):
```

```

        return float(np.clip(0.40*reid + 0.30*u + 0.20*(1-min(k,10)/10) + 0.10*Hn, 0, 1))

# --- run ---
synd = extract("synd_mains_1min.csv", "SynD")
real = pd.concat([extract(f"snm_{b}_mains_1min.csv", b)
                  for b in ["b10", "b11", "b15", "b17"]], ignore_index=True)
su,sH,sk = uniqueness_entropy_k(synd, household_level=False)
ru,rH,rk = uniqueness_entropy_k(real, household_level=True)
rreid,_ = single_day_reid(real)
print("SynD PRS:", prs(su,sH,sk, 0.0))
print("Real PRS:", prs(ru,rH,rk, rreid), " single-day re-id:", rreid)

```

Appendix Listing. Consolidated, reproducible analysis pipeline.

Data preparation note: SynD mains was down sampled with Listing 2; each SNM home's mains was computed from per-phase V/I/PF and down sampled with Listing 3; archives were handled with the 7-Zip commands in Listing 1. All processing used Python 3.12 on Windows 10.

Acknowledgement of AI Assistance

This report was partially supported by the use of AI tools: Claude.ai, for guidance and clarification on selected challenges including the python codes.

All other analysis, interpretations, and final content are my own work.